

Olufemi Taiwo

Data Science Engineer 4 - API & Data Quality Lead

GitHub: (To be assigned)

Access Level: Maintainer

Reports To: @Goldokpa (Project Owner)

Project Duration: 3 Months

How This Role Fits You

Your current role as Reporting and Data Quality Officer at the Royal Marsden NHS Foundation Trust is the clearest signal for this assignment. Every working day you validate data flows, investigate mismatches across Epic EPR, troubleshoot system errors using SQL, and hold the line on reporting accuracy for senior clinical stakeholders. That obsessive attention to data integrity at every step from input to output is exactly what ClimateVision's API and inference pipeline need.

At Fidelity Bank, you kept payment platforms reliable around the clock as an Application Support Analyst - triaging incidents, analysing root causes, and producing service reports that guided operational decisions. ClimateVision runs a similar system: satellite images arrive as requests, the API must respond correctly and quickly, and any failure needs to be caught, logged, and escalated before it reaches users. That is your wheelhouse.

Your Business Intelligence work at Dataleum - building Power BI dashboards, conducting data quality checks, achieving 98% GDPR compliance - means you already understand auditability. In a climate monitoring system used by NGOs and government agencies, every prediction must be traceable, every alert explainable, and every data flow compliant. You build that confidence layer.

Your ITIL 4 certification is a direct fit for incident management, change control, and problem management in production. Combined with your MSc in Data Science, you are the person who makes the API not just functional, but operationally trustworthy - with structured logging, audit trails, validated schemas, and monitoring that surfaces issues before users notice them.

Your Role on ClimateVision

You own the API layer and the inference pipeline - everything between a trained model and a user receiving a validated, structured response. You ensure the system is reliable, observable, and produces outputs that are correct and auditable. You are the data quality gatekeeper for every prediction that leaves the system.

Core Responsibilities

- Extend and maintain the FastAPI backend (endpoints, authentication, request validation)
- Build Pydantic schemas for all API request/response objects - the contract for data quality
- Implement structured logging, error handling, and audit trails throughout the inference flow
- Build the inference validation layer - catch bad inputs, validate outputs, flag anomalies
- Create the deforestation alert system with configurable thresholds and notification routing
- Build API monitoring endpoints: health checks, data quality metrics, run status dashboards

- Write SQL queries and admin endpoints for operational reporting and data audits
- Design and document the API contract (request/response schemas, error codes, versioning)

Your Codebase Ownership

You are the primary owner of the following files and directories:

src/climatevision/inference/	# PRIMARY OWNER
pipeline.py	# Core inference pipeline
batch_processor.py	# Batch processing with job queuing
postprocess.py	# Output filtering & thresholding
alert_generator.py	# Deforestation alert system
__init__.py	
src/climatevision/api/	# PRIMARY OWNER
main.py	# FastAPI application
auth.py	# API key authentication
middleware.py	# Request logging, CORS
schemas.py	# Pydantic request/response schemas
__init__.py	
src/climatevision/db.py	# CO-OWNER - Database & audit queries
run_api.sh	# API startup script
config.yaml	# API & inference config sections

Your 3-Month Delivery Timeline

MONTH 1: Foundation (Weeks 1-4)

Week 1-2: Schemas & Validation

- Build schemas.py - Pydantic models for every API request and response object
- Extend pipeline.py with input validation: image shape, band count, coordinate bounds
- Add structured JSON logging throughout the inference flow (request ID, timestamps, errors)
- Implement output validation - flag predictions outside expected confidence ranges

Week 3-4: API Hardening

- Implement auth.py - API key authentication and organisation-based access control
- Build middleware.py - request logging, CORS, request size limits
- Create /api/health, /api/status, and /api/metrics endpoints for operational monitoring
- Write API integration tests covering validation edge cases and error responses

MONTH 2: Quality & Alerts (Weeks 5-8)

Week 5-6: Inference Quality Layer

- Build postprocess.py - confidence thresholding and prediction filtering

- Implement anomaly detection for unusual inference outputs (flag for review)
- Create audit log entries for every prediction: input hash, model version, output summary
- Build `batch_processor.py` - parallel image processing with per-job status tracking

Week 7-8: Alert System & Reporting

- Build `alert_generator.py` - configurable deforestation threshold alerting
- Implement notification routing (email, webhook) for triggered alerts
- Write SQL reporting queries for run history, error rates, and data quality KPIs
- Create admin endpoints for operational dashboards: throughput, failure rates, alert volumes

MONTH 3: Observability & Documentation (Weeks 9-12)

Week 9-10: Monitoring & Data Quality Reports

- Build a `/api/reports` endpoint returning data quality metrics over configurable time windows
- Implement request tracing: correlate API requests to inference runs to alerts
- Create a data quality dashboard feed (JSON) for the frontend to visualise pipeline health
- SQL-based audit trail queries: who requested what, when, and with what result

Week 11-12: Documentation & Launch Readiness

- Write the API reference: all endpoints, schemas, error codes, and usage examples
- Document the incident response runbook: what each error means and how to resolve it
- Security review: input sanitisation, SQL injection checks, API key rotation procedures
- Final integration testing with all team modules - validate end-to-end data flow

Your Git Workflow

```
# Create feature branches from develop
git checkout develop
git pull origin develop
git checkout -b feature/api-schemas

# Your branch naming convention:
feature/api-*           (API features & endpoints)
feature/inference-*     (inference pipeline & validation)
feature/schemas-*      (Pydantic schema changes)
fix/api-*               (bug fixes)
```

All PRs go to the develop branch. Tag @cutewizzy11 for API contract reviews (he consumes your endpoints from the frontend) and @edoh-Onuh when touching inference logic that involves model outputs.

Your Key Collaborators

- @edoh-Onuh (ML Lead) - Their trained models are loaded by your inference pipeline. Coordinate on model format (.pth vs ONNX), input shapes, output schemas, and confidence score formats.
- @Oshgig (Data Pipeline Lead) - Your inference input validation must match her preprocessing exactly. Align on

normalization constants, expected band order, and coordinate formats.

- @franchise (Analytics Lead) - Their analytics endpoints are exposed through your API. Coordinate on response schemas, pagination, and data quality flags in outputs.
- @cutewizzy11 (Full-Stack & CI/CD) - He consumes your API from the frontend and manages Docker and deployment. You two define the API contract together - endpoints, schemas, error codes.

Your Code Pipeline

Your pipeline covers running and validating the FastAPI server, testing all endpoints, enforcing data quality, and maintaining the inference layer.

Step 1: Environment Setup

```
git clone https://github.com/Climate-Vision/ClimateVision.git
cd ClimateVision
pip install -r requirements.txt

# Set environment variables
cp .env.example .env
# Edit .env: set MODEL_PATH, DB_PATH, API_KEY_SECRET
```

Step 2: Start the API Server

```
# Start FastAPI in development mode with auto-reload
uvicorn climatevision.api.main:app \
  --reload \
  --host 0.0.0.0 \
  --port 8000

# Interactive API docs available at:
# http://localhost:8000/docs
# http://localhost:8000/redoc
```

Step 3: Test Prediction Endpoints

```
# Test JSON prediction endpoint
curl -X POST http://localhost:8000/predict/json \
  -H "Content-Type: application/json" \
  -d '{
    "bbox": [-60, -15, -45, 5],
    "start_date": "2023-01-01",
    "end_date": "2023-12-31",
    "analysis_type": "deforestation"
  }'

# Test file-upload endpoint
curl -X POST http://localhost:8000/predict/upload \
  -F "file=@data/test/sample_tile.tif" \
  -F "analysis_type=flooding"

# Health check
```

```
curl http://localhost:8000/health
```

Step 4: Run Data Quality Checks

```
# Validate all run records in the database meet schema requirements
python - <<'EOF'
from climatevision.db import get_db_connection, validate_run_schema
conn = get_db_connection()
issues = validate_run_schema(conn)
if issues:
    print(f'Data quality issues found: {len(issues)}')
    for issue in issues:
        print(f' - {issue}')
else:
    print('All records pass quality checks')
EOF
```

Step 5: Register an NGO Organisation

```
# Create an NGO organisation via the API
curl -X POST http://localhost:8000/organizations \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Amazon Conservation Trust",
    "email": "alerts@amazonconservation.org",
    "region": "amazon"
  }'

# Add a regional monitoring subscription
curl -X POST http://localhost:8000/organizations/1/subscriptions \
  -H "Content-Type: application/json" \
  -d '{"bbox": [-60, -15, -45, 5], "analysis_type": "deforestation", "alert_threshold": 0.15}'
```

Step 6: Commit & Push Your Work

```
# Switch to your git identity
source team_docs/switch_user.sh olufemi

git checkout develop && git pull origin develop
git checkout -b feature/api-input-validation

git add src/climatevision/api/main.py
git add src/climatevision/db.py
git commit -m "feat(api): add Pydantic input validation and audit logging to predict endpoints"

# Push from YOUR GitHub account (femi23)
git push olufemi feature/api-input-validation
```